Index

V&V Automation Testing (Playwright) Course Structure

Playwright+Javascript		
Sr. No.	**Course**	**Duration (Days)**
1	Onboarding Activities	1
2	L&D, HR & Ops Orientation	1
3	Discover	1
4	Power Skills (Behavioural)-Foundation-Session 1	6
5	Web Basics (HTML 5, CSS 3, XML)	3
6	DBMS/SQL	2
7	JavaScript, TypeScript , Node js	7
8	Fundamental Assessment (Coding Test)	0.5
9	Testing Concepts + Use Case	5
10	Introduction to Agile	0.5
11	Defect Reporting and JIRA	1
12	Playwright Automation	6
13	Sprint 1 Implementation	6
14	Sprint 1 Evaluation	1
15	L1 Preparation	1
16	Generative AI 101	0.5
17	L1 Assessment (MCQ based assessment)	0.5
18	Total	40

V&V Automation Testing (Playwright) TOC

Web Basics (HTML5, CSS3, XML)

Program Duration: 3 days

Contents:

- HTML Basics
    - Understand the structure of an HTML page.
    - New Semantic Elements in HTML 5
    - Learn to apply physical/logical character effects.
    - Learn to manage document spacing.
- Tables
    - Understand the structure of an HTML table.
    - Learn to control table format like cell spanning, cell spacing, border
- List
    - Numbered List
    - Bulleted List
- Working with Links
    - Understand the working of hyperlinks in web pages.
    - Learn to create hyperlinks in web pages.
    - Add hyperlinks to list items and table contents.
- Image Handling
    - Understand the role of images in web pages
    - Learn to add images to web pages
    - Learn to use images as hyperlinks
- Frames
    - Understand the need for frames in web pages.
    - Learn to create and work with frames.
- HTML Forms for User Input
    - Understand the role of forms in web pages
    - Understand various HTML elements used in forms.
    - Single line text field
    - Text area
    - Check box
    - Radio buttons
    - Password fields
    - Pull-down menus
    - File selector dialog box
- New Form Elements
    - Understand the new HTML form elements such as date, number, range, email, search and datalist
    - Understand audio, video, article tags
- Introduction to Cascading Style Sheets 3.0
    - What CSS can do
    - CSS Syntax
    - Types of CSS
- Working with Text and Fonts
    - Text Formatting
    - Text Effects
    - Fonts
- CSS Selectors
    - Type Selector

- Universal Selector
  - ID Selector
  - Class selector
- Introduction to XML
  - Evolution of XML
  - Role of XML in Web Applications
  - Different members of XML family
  - Introduction to Namespace

JavaScript, TypeScript & Node js for V&V Automation Testing

Program Duration: 8 days.

Contents:

- Introduction to JavaScript
  - Introduction to JavaScript
  - Features of JavaScript
  - Evolution of JavaScript
  - Developing software in JavaScript
- Setting up JavaScript Development Environment
  - Installing Node.js
  - Choosing and Setting up an IDE (Visual Studio Code, Sublime Text)
  - Introduction to NPM (Node Package Manager)
  - Using the Terminal/Command Line for JavaScript development
- JavaScript Language Fundamentals
  - Keywords and Reserved Words
  - Data Types in JavaScript (Primitive & Non-Primitive)
  - Variables (let, const, var)
  - Operators and Assignments
  - Flow Control: if, switch, loops
  - Functions and Arrow Functions
  - Best Practices in Language Fundamentals
- Objects and Arrays
  - Introduction to Objects
  - Creating and Using Objects
  - Arrays and Array Methods
  - Iterating Over Arrays (for, forEach, map, filter)
  - Object-Oriented Concepts in JavaScript (Prototypes, Inheritance)
  - Best Practices for Objects and Arrays
- JavaScript Functions and Scope
  - Function Declarations and Expressions
  - Parameters and Arguments
  - Understanding Scope (Global, Local, Lexical Scope)
  - Closures and their Uses
  - The 'this' Keyword
  - Best Practices for Functions and Scope
- Asynchronous JavaScript
  - Understanding Callbacks

    - Promises and Promise Chaining
    - Async/Await
    - Error Handling in Asynchronous Code
    - Event Loop and Non-blocking I/O
    - Best Practices in Asynchronous Programming
- Error Handling in JavaScript
    - Introduction to Error Handling
    - try-catch-finally
    - Throwing Errors
    - Custom Errors
    - Best Practices for Error Handling
- JavaScript Classes and Inheritance
    - Introduction to Classes
    - Constructors and Methods
    - Inheritance and the 'extends' keyword
    - Superclass and Subclass
    - Getters and Setters
    - Best Practices in Classes and Inheritance
- Regular Expressions in JavaScript
    - Introduction to Regular Expressions
    - Basic Regular Expression Syntax
    - Using RegExp Objects
    - Validating Data with Regular Expressions
    - Best Practices for Regular Expressions
- JavaScript for Testing Automation
    - Introduction to Testing Frameworks (Mocha, Jest, Jasmine)
    - Setting up and Running Tests with Mocha
    - Writing Unit Tests in JavaScript
    - Testing Asynchronous Code
    - Mocking and Stubbing
    - Best Practices for Testing
- ES6 & Typescript
    - Var, Let and Const keyword
    - Arrow functions, default arguments
    - Template Strings, String methods
    - Object de-structuring
    - Spread and Rest operator
    - Typescript Fundamentals
    - Types & type assertions, Creating custom object types, function types
    - Typescript OOPS - Classes, Interfaces, Constructor, etc
    - Asynchronous Programming in ES6
    - Promise Constructor
    - Promise with Chain
    - Promise Race
- Node js
    - Introduction to Node js
    - Installation of Node js
    - Basics of Node js
    - First Application of node js
    - Node js NPM
    - Node js module
    - Node js Assertion module

Testing Concepts (STLC, Defect Life Cycle, Testing Processes + Use Case)

Program Duration: 4.5 days

Contents:

- Fundamentals of Testing
    - Introduction to Software Testing
    - Software Testing - Definitions
    - Need of Software Testing
    - Error-Failure-Defect
    - Causes of Software Defects
    - Cost of Software Defects
    - What does Software Testing reveal?
    - Importance of Software Testing
    - Importance of Testing Early in SDLC Phases
    - Testing and Quality
    - Quality Perception
    - Seven Testing Principles
    - Economics of Testing
    - How Testing is conducted?
    - Software Testing – Then (Past)
    - Software Testing – Now (Present)
    - Scope of Software Testing
    - Factors influencing the Scope of Testing
    - Risk Based Testing
    - Project Risks
    - Product Risks
    - Need of Independent Testing
    - Activities in Fundamental Test Process
    - Attributes of a good Tester
    - Psychology of Testing
    - Code of Ethics for Tester
    - Limitations of Software Testing
- Testing Throughout the Software Development Lifecycle
    - Software Development Lifecycle Models
    - Software Development and Software Testing
    - Software Development Lifecycle Models in Context
    - Test Levels
    - Component Testing
    - Integration Testing
    - System Testing
    - Acceptance Testing
    - Test Types
        - Functional Testing
        - Non-functional Testing
    - White-box Testing
    - Smoke & Sanity Testing
    - Change-related Testing

- Test Types and Test Levels
- Maintenance Testing
- Triggers for Maintenance
- Impact Analysis for Maintenance
- Test Case Terminologies
- Test Data

- Static Testing
  - Types of Testing Techniques
  - Differences between Static and Dynamic Testing
  - Static Testing Basics
  - Work Products that Can Be Examined by Static Testing
  - Benefits of Static Testing
  - Review Process
  - Work Product Review Process
  - Roles and responsibilities in a formal review
  - Review Types
  - Applying Review Techniques
- Test Techniques
  - Categories of Test Techniques
  - Choosing Test Techniques
  - Categories of Test Techniques and Their Characteristics
  - Black-box Test Techniques
  - Equivalence Partitioning
  - Boundary Value Analysis
  - Decision Table Testing
  - State Transition Testing
  - Use Case Testing
  - White-box Test Techniques
  - Statement Testing and Coverage
  - Decision Testing and Coverage
  - The Value of Statement and Decision Testing
  - Experience-based Test Techniques
  - Error Guessing
  - Exploratory Testing
  - Checklist-based Testing
- Test Management & Test Metrics
  - Test Organization
  - Independent Testing
  - Tasks of a Test Manager and Tester
  - Test Planning and Estimation
  - Purpose and Content of a Test Plan
  - Test Strategy and Test Approach
  - Entry Criteria and Exit Criteria (Definition of Ready and Definition of Done)
  - Test Execution Schedule
  - Factors Influencing the Test Effort
  - Test Estimation Techniques
  - Test Monitoring and Control
  - Metrics Used in Testing
  - Purposes, Contents, and Audiences for Test Reports
  - Configuration Management
  - Risks and Testing
  - Definition of Risk
  - Product and Project Risks
  - Risk-based Testing and Product Quality

- Defect Management Testing Metrics

- Requirement Engineering
  - Evolution of Requirements
  - Who provides the Requirements?
  - Challenges in Requirement Gathering
  - Why do we need good requirements?
  - Characteristics & Impact of bad Requirements
  - Requirement Engineering
  - Functional Vs Non-Functional Requirements
  - Non Functional Requirements: FURPS +
  - Stable and Volatile Requirements
  - Baselining Requirements
  - Requirements Traceability
  - Requirement Traceability Matrix
  - Maintaining Requirement Traceability
  - Requirement Traceability Matrix – Example
  - Requirements Change
  - Change Management Process
  - Requirement Creep
- Use Case Testing
  - Use case modeling
  - Advantage of use cases
  - Actor
  - Goals and Requirements
  - Goals and scenarios
  - Naming Conventions
  - Alternate Path
  - Exceptions
  - Errors
  - Precondition & Postcondition
  - Good practices
  - Failure scenarios
- Software Version Guidance
  - Introduction to Software Versioning
  - Major Release
  - Minor Release
  - Revision Release
  - Build Release
  - Beta Version for User Testing

Introduction to Agile

Program Duration: 0.5 day.

Contents:

- Agile Process Framework
  - History of Traditional Software Development Model
  - Software Development Model and SDLC
  - “Waterfall Model” – An Overview
  - Waterfall or Sequential Based Development Model
  - “Real Life” – Waterfall Model

- “Waterfall Model” – Advantages
- “Waterfall Model” – Disadvantages
- Agile Software Development – Definition
- Agile Development Model
- Graphical Illustration of Agile Development Model
- Why use Agile?
- Agile Manifesto and Principles
- 12 Principles of Agile Methods
- Agile Values
- What is NOT an Agile software development?
- Foundation of an Agile software development Method
- Common Characteristics of Agile Methods
- Agile Methods and Practices
- When to use Agile Model?
- Advantages of Agile Model
- Disadvantages of Agile Model
- Difference between Agile and Waterfall Model
- Agile – Myths and Reality
- Agile Market Insight

- Agile Methods and Practices - SCRUM
  - Introduction to SCRUM
  - Scrum Roles and Responsibilities
  - Scrum Core Practices and Artifacts
    - User Story
    - Sprint
    - Release Planning Meeting
    - Sprint Planning Meeting
    - Daily Scrum Meeting (Daily Stand up)
    - Sprint Review Meeting
    - Retrospective
    - Product Backlog
    - Sprint Backlog
    - Burn-Down Chart
    - Velocity
    - Impediment Backlog
  - Definition of “Done”
  - Splitting User Story into Task
  - Why to Split User Story into Task?
  - Guidelines for Breaking Down a User Story into Tasks
  - Examples of Scrum Task Board
  - Planning Poker®
  - Planning Poker® - Process/Steps
  - What are Story Points?
  - How do We Estimate in Story Points?
  - What Goes into Story Points?
- Agile Methods and Practices - Extreme Programming (XP), Lean Software Development & Kanban
  - Introduction to Extreme Programming
  - The Rules of Extreme Programming
  - Extreme Programming (XP) - Principles
  - Extreme Programming (XP) – Key Terms
  - Introduction to Lean Software Development
  - Principles of Lean Software Development
  - What is Kanban?

- Introduction to Agile Testing
    - What is Agile Testing?
    - Agile Team - Roles and Activities
    - Where does Tester fit in Agile Team?
    - Agile Team – Tester's Role and Responsibilities
    - Agile Team – Test Manager's Role and Responsibilities
    - How is Agile Testing different?
    - Traditional Testing Vs. Agile Testing
    - What is Iteration 0?
    - User Story Perspective Agile Testing Process
    - Tester's Change in Mind-Set – A key to success
- Agile Testing Quadrants and Agile Test Planning
    - An overview of Agile Testing Quadrants
    - Agile Testing Quadrant 1, 2 & 3 Goals
    - Agile Testing Quadrant 1, 2 & 3 Toolkit
    - Test Planning in Agile Testing

Defect Reporting and JIRA

Program Duration: 1 day.

Contents:

- Defect Free Defect Reporting
    - What is Software Quality?
    - Defect Definition
    - Why find Defects?
    - Impact of Defects
    - Legal Implications
    - Life-cycle workflow
    - Life-cycle workflow - Enhancement
    - Defect Report - Definition
    - Defect Reporting – The Need
    - Defect Report - Template
    - Important Attributes
    - Example on Severity & Priority
    - Defect Report Users
    - Defect Management
    - Logging, Tracking & Analysis
    - What is Defective Defect Report?
    - The reasons for defective reports
    - Other Influencing Factors
    - The Most Severe Defect
    - Importance of Effective Defect Reporting
- Overview of Defect Tracking Tools
    - Introduction to Defect Tracking Tool
    - Introduction to Bugzilla
    - Features of Bugzilla
    - Bugzilla – User Interface

    - Introduction to JIRA
    - Introduction to HP ALM
    - Features of HP ALM
    - HP ALM – User Interface
- Introduction to JIRA
    - Overview of JIRA
    - Features of JIRA
    - JIRA Users
    - JIRA Software Workflow
    - Basic Concepts of JIRA
    - Issue, Project, Workflow, Components & Versions
    - JIRA Software
    - Overview on Issues
    - Priority of Issues
    - Issue Workflow
    - Overview of Projects
    - Sub tasks
- Hands-on JIRA
    - Login to JIRA
    - Create a SCRUM Project,
    - Inviting team members,
    - Creating components
    - Creating Issues – Epics, Stories, Tasks, Bugs
    - Creating sub-tasks
    - Managing Issues, issue properties.
    - Importing Issues from csv file into JIRA
    - Configuring ZEPHYR for Test Case Management
    - Create Sprints in JIRA & Issue Workflow
    - JIRA Reports – Epic report, created vs Resolved Issues Report, sprint report, Burn down chart, Velocity chart.

Test Automation with Playwright

Program Duration: 7 days.

Contents:

- Introduction to Playwright
    - Overview of Playwright
    - Advantages of Playwright over other automation tools
    - Playwright's Ecosystem and Architecture
    - Use Cases of Playwright
- Installation and Setup
    - Setting up Playwright on Windows
    - Setting up Playwright on macOS
    - Setting up Playwright on Linux
    - Verifying Playwright Installation
    - Installing and Managing Dependencies with npm/yarn

- Creating and Running Tests
    - Writing Basic Playwright Tests
    - Running Playwright Tests from the Command Line
    - Understanding Playwright Test Results
    - Using Playwright Test Runner
    - Configuring Playwright for different environments (headless, headed)
- Selectors
    - Using CSS Selectors
    - Using XPath Selectors
    - Playwright-Specific Selectors (e.g., text, role, etc.)
    - Robust Selectors for Dynamic Web Pages
    - Chained Selectors and Complex Queries
    - Best Practices for Selecting Elements
- Interacting with Elements
    - Clicking Elements
    - Typing Text and Input Handling
    - Selecting Options in Dropdowns
    - Drag and Drop Interactions
    - Hover, Focus, and Other Events
    - Scrolling and Page Interactions
- Assertions
    - Using Playwright's Built-in Assertion Library
    - Common Assertions in Playwright (e.g., element visibility, text content, etc.)
    - Enhanced Assertions with expect API
    - Text and Attribute Assertions
    - Custom Assertions for Specific Test Scenarios
- Navigation
    - Navigating Between Pages
    - Handling Page Redirects
    - Waiting for Elements or Pages to Load
    - Controlling Browser Navigation with goBack(), reload(), waitForNavigation()
    - Handling Network Events During Navigation
- Frames and Iframes
    - Working with Frames and Iframes in Playwright
    - Locating and Interacting with Elements Inside Frames
    - Switching Between Frames and Iframes
    - Handling Nested Frames
- Handling Dialogs
    - Managing Alerts, Prompts, and Confirmations
    - Intercepting and Automating Dialogs in Playwright
    - Dismissing or Accepting Alerts
    - Extracting Text from Dialogs
- Network Interception
    - Mocking Network Requests with Playwright
    - Modifying Responses to Simulate Real-World Scenarios
    - Monitoring Network Activity During Tests
    - Intercepting and Blocking Requests
    - Testing API Calls and Responses with Playwright
- Visual Comparisons
    - Taking Screenshots of Pages or Elements
    - Comparing Screenshots for Visual Regression Testing

    - Integrating Playwright with Visual Regression Tools
    - Best Practices for Visual Comparison in Playwright
- Multi-browser Testing
    - Running Tests in Different Browsers (Chromium, Firefox, WebKit)
    - Cross-Browser Testing with Playwright
    - Handling Browser-Specific Behavior
    - Playwright and Browser Contexts
- Parallel Test Execution
    - Running Tests in Parallel to Speed Up Test Execution
    - Configuring Playwright to Run Tests Concurrently
    - Understanding Test Isolation in Parallel Execution
    - Managing Resource Conflicts in Parallel Runs
- Playwright Test (Native Test Runner)
    - Introduction to Playwright Test Runner
    - Setting Up Playwright Test
    - Running Tests with Playwright Test
    - Using Test Hooks in Playwright
    - Handling Test Failures and Retrying Tests
- Data-Driven Testing
    - Parameterizing Tests with Different Data Sets
    - Using CSV, Excel, or JSON for Data-Driven Testing
    - Data-Driven Test Automation with Playwright
    - Integrating Playwright with External Data Sources